Software Design Document

StructSure - Infrastructure degradation manager

Version 1.0

StructSure Team:
Andres Stephen Lopez
Nicole Fonseca
Logan Pritchard
Grace Hechevarria

Professor: Charlyne Walker

Florida International University

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Andres Stephen Lopez Nicole Fonseca Logan Pritchard Grace Hechavarria	Initial Release	January 18, 2026

Contents

1 Introduction

1.1 Purpose

The Software Design Document describes the architecture and system design for StructSure - Infrastructure Degradation Management Platform. This document is intended for Project Managers, Software Engineers, and anyone else who will be involved in the implementation of the system.

1.1.1 Stakeholders and Context

Primary users include residents/tenants who report infrastructure issues, property associations and building management companies who need to be held accountable, third-party organizations monitoring infrastructure, and municipalities tracking building conditions and compliance. Property associations often claim they had no knowledge of infrastructure issues or that they couldn't process all the information in time. They need a centralized system where issues are documented, tracked, and they receive persistent notifications. Residents need an easy way to report issues, and municipalities/third parties need visibility into building conditions and accountability records.

1.1.2 Key Features and Differentiation

Unlike generic social media platforms or complaint systems, this tool combines Instagram-like ease of use with specific accountability features. The application requires real-time photo capture using only the camera (no gallery access) to prevent spoofing and defamation. Photos are automatically linked to locations via geo-location services. The system uses Google Places API to identify nearby buildings, allowing users to select from 2-3 nearby buildings if the location is imprecise. Users have blog-style accounts without real name requirements. Each building maintains a record of all posts related to it. The platform automatically sends persistent notifications and emails to associations using publicly available contact information. This design prevents mass uploading of old/edited images and fake accounts from posting preset galleries to damage a building's reputation.

1.1.3 Expected Impact

The tool creates a public record of infrastructure issues tied to specific buildings, making it difficult for associations to claim ignorance. It enables accountability through persistent notifications and public visibility. Municipalities and third parties can track building conditions and compliance. This transparency may lead to faster response times from associations and improved infrastructure maintenance overall.

1.2 Scope

This document describes the implementation details of the Infrastructure Degradation Management Platform. The system will consist of six major components: Photo Capture, Build-

ing Management, Notification System, Database, Authentication, and Public Access. Each of the components will be explained in detail in this Software Design Document.

1.3 Definitions, Acronyms and Abbreviations

Acronym	Meaning
SDD	Software Design Document
API	Application Programming Interface
EXIF	Exchangeable Image File Format
GPS	Global Positioning System
SDK	Software Development Kit

1.4 References

- Google Places API Documentation
- Misskey GitHub Repository: https://github.com/misskey-dev/misskey
- Expo Camera Documentation: https://docs.expo.dev/versions/latest/sdk/camera/
- Cloudinary Documentation: https://cloudinary.com/documentation/image_upload_api_reference
- SendGrid API Documentation: https://docs.sendgrid.com/api-reference
- Firebase Authentication Documentation: https://firebase.google.com/docs/auth
- Cursor IDE: https://cursor.sh/ - AI-powered code editor used for development and documentation
- Anthropic Claude (via Cursor): Large Language Model AI assistant used for code generation, documentation, and design assistance throughout the project
- LaTeX Project: https://www.latex-project.org/ - Document preparation system used for creating this Software Design Document

2 System Overview

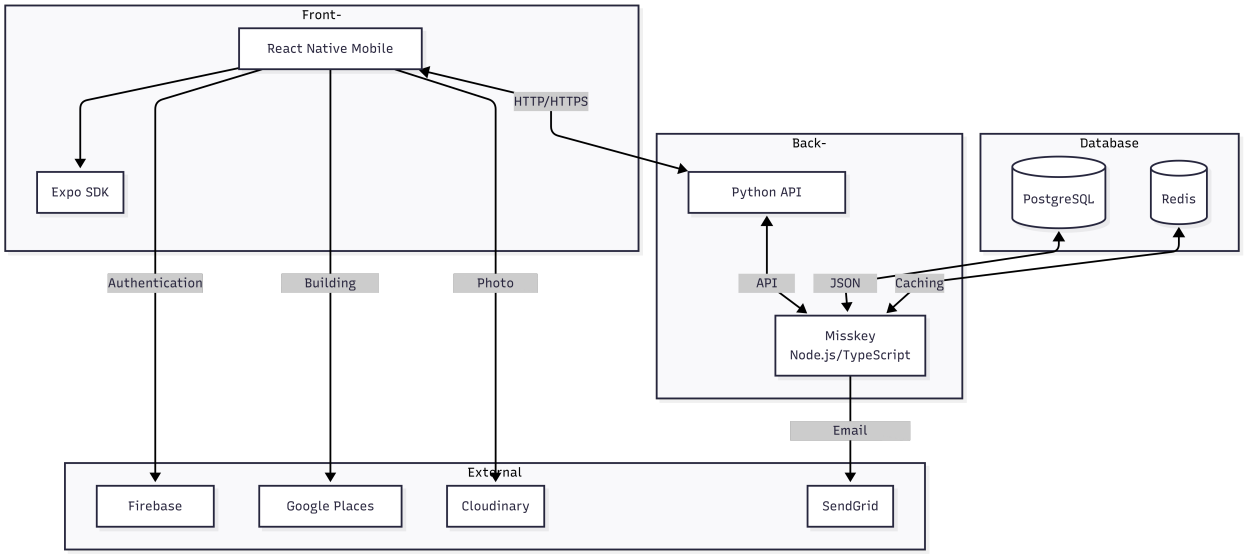

Figure 1: System Architecture Diagram

Figure 1 above represents the architectural structure chosen for the development of the Infrastructure Degradation Management Platform. The system follows a three-tier architecture consisting of a database layer, back-end layer, and front-end layer.

The database layer utilizes PostgreSQL for persistent data storage of users, buildings, and reported issues, while Redis provides caching capabilities for improved performance. The back-end layer is built on Misskey, a Node.js and TypeScript-based social networking platform that provides fast, efficient infrastructure with built-in user management, post/timeline features, and database integration. To accommodate team members more familiar with Python, a Python API wrapper interfaces with the Misskey backend, allowing Python-based development while maintaining the core Misskey functionality.

The front-end layer consists of a React Native mobile application built with the Expo SDK, which provides camera and location APIs for photo capture and GPS coordinate retrieval. The mobile app communicates with the back-end through the Python API wrapper via HTTP/HTTPS requests.

The system integrates with several external services: Cloudinary for photo storage and processing, Google Places API for building identification based on GPS coordinates, Firebase Authentication for user account management, and SendGrid for email notifications to property associations. Currently, the notification system sends emails to associations when new issues are reported, though this can be extended to support additional notification methods in the future.

Storing information about users, buildings, and reported issues enables the system to maintain building records, track issue timelines, and manage notification history. User information enables the system to maintain user profiles and keep records of their reported issues.

Additionally, by following this architecture and structure, the system can be extended into additional features such as municipality dashboards, public record exports, and enhanced notification systems beyond email. However, this is out of the scope of this project and will be addressed in a future version.

3 System Components

3.1 Decomposition Description

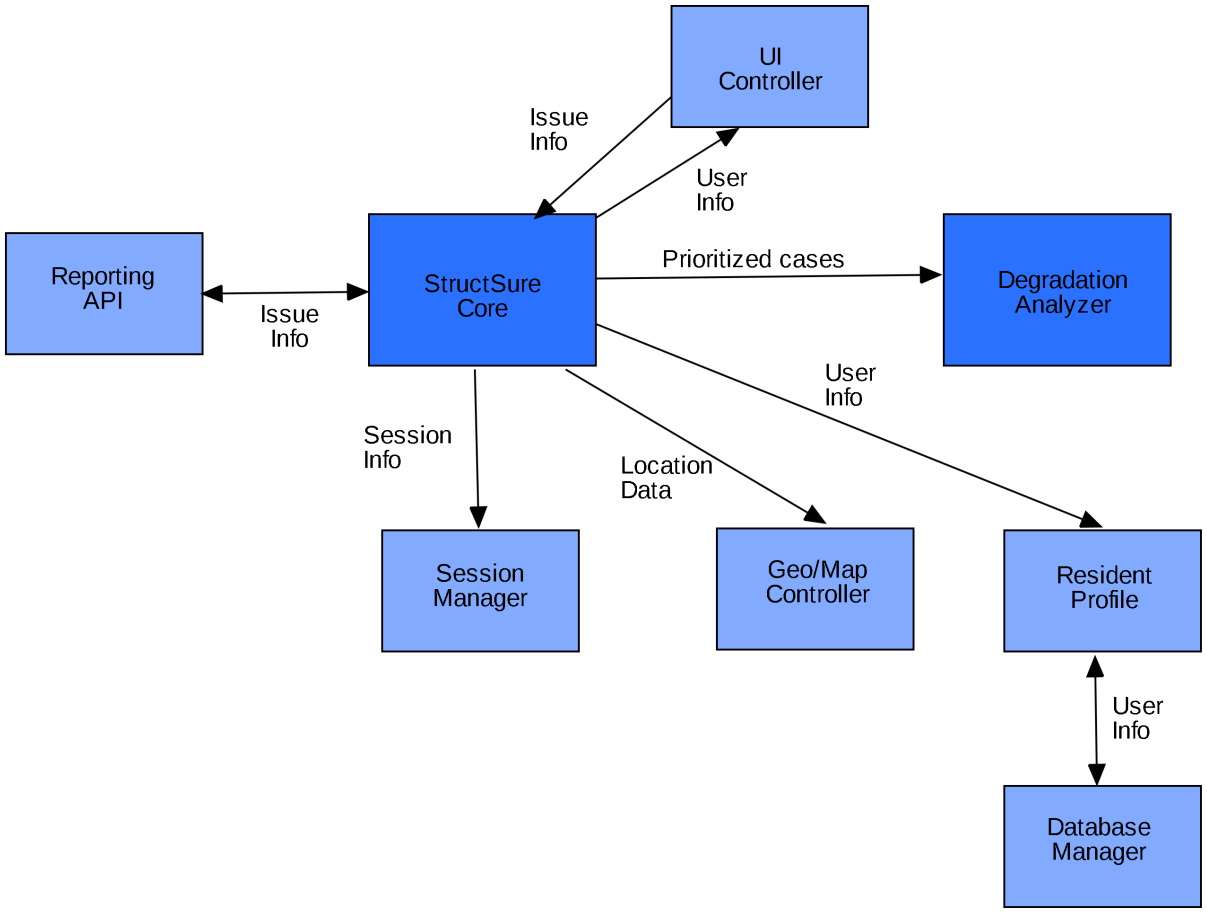

Figure 2: System Decomposition Diagram

Figure 2 above shows a top-down description of how the application is expected to work and how components will interact with one another. This logical component view complements the technology implementation view described in Section 2. The system consists of several key components:

The **UI Controller** is the main entry point for users, implemented as a React Native mobile application with the Expo SDK. It provides methods for capturing photos/videos,

attaching location data, submitting reports, viewing status, and managing notifications.

The **StructSure Core** serves as the central orchestrator, implemented using the Misskey backend framework with a Python API wrapper. It coordinates interactions between all system components and manages the overall application flow.

The **Reporting API** handles report management and is implemented as part of the Misskey backend with Python API interfaces. It provides methods for creating reports, updating reports, adding evidence, listing reports, and exporting/sharing reports.

The **Session Manager** manages user authentication, implemented using Firebase Authentication. It provides methods for sign up, login, logout, and user validation.

The **Geo/Map Controller** handles geographical and map-related operations, integrating with the Google Places API. It provides methods to render maps, add markers, and show directions for building identification.

The **Resident Profile** manages resident-specific data including identity, address, association information, and notification preferences. This component utilizes the user management capabilities of Misskey.

The **Degradation Analyzer** performs analysis on reported issues to assess severity, detect duplicates, and prioritize cases. This component implements custom logic within the Misskey backend to analyze infrastructure degradation patterns.

The **Database Manager** handles all data persistence, implemented using PostgreSQL for persistent storage and Redis for caching. It provides methods to add/update users and reports, retrieve reports, and delete reports.

3.2 Dependency Description

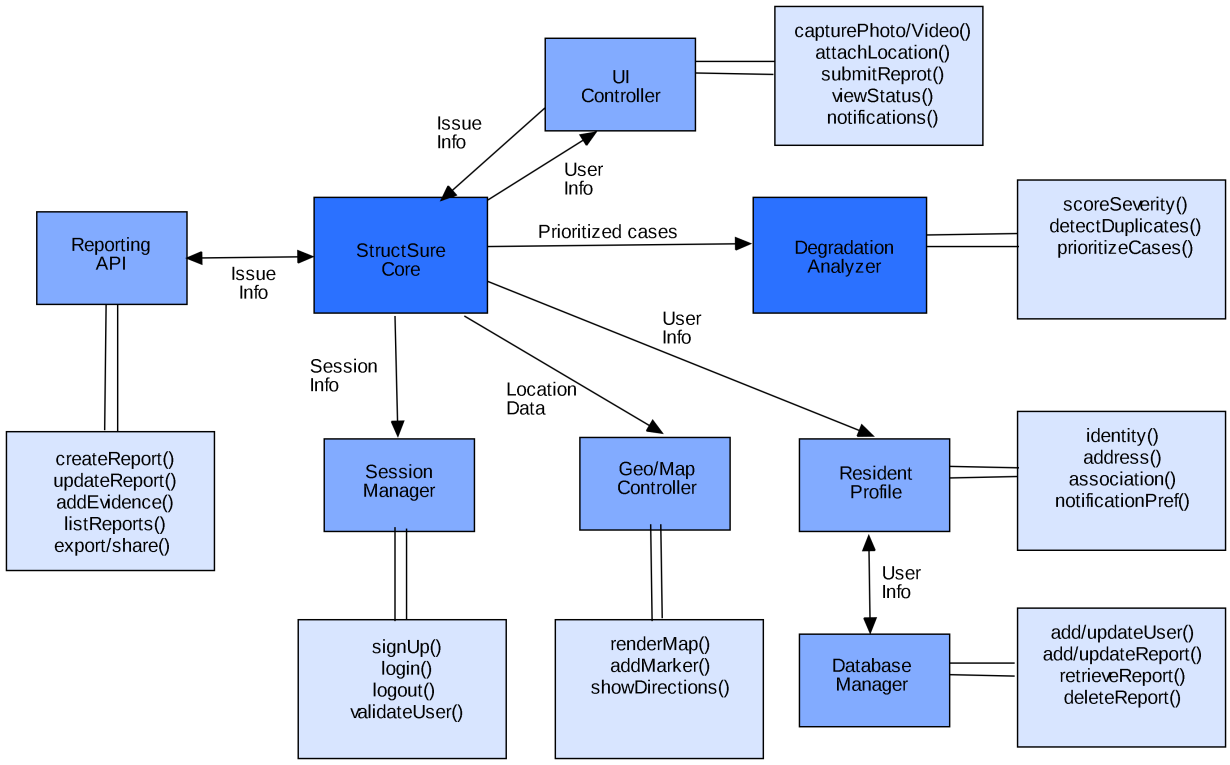

Figure 3: System Dependency Diagram

Figure 3 above represents the component diagram of the system and how each module is dependent on another for its functionality. This diagram illustrates the logical flow and dependencies between components, which are implemented using the technology stack described in Section 2.

The UI Controller, implemented as a React Native mobile application, initiates user interactions and communicates with the StructSure Core, which serves as the central processing unit implemented using Misskey with a Python API wrapper. The StructSure Core orchestrates interactions with specialized components: the Reporting API for managing issue reports, the Session Manager (Firebase Authentication) for user authentication, the Geo/Map Controller (Google Places API integration) for location and mapping services, and the Degradation Analyzer for analyzing reported infrastructure issues.

The Session Manager handles user authentication through Firebase Authentication and communicates directly with the Database Manager (PostgreSQL) to store and retrieve user session data. The Geo/Map Controller processes location information through the Google Places API and stores geographical data in the Database Manager. The Degradation Analyzer processes reported issues and interacts with the Resident Profile component, which manages resident information using Misskey's user management features and also stores data in the Database Manager.

The Reporting API, implemented through the Misskey backend and Python API wrapper, creates report posts that are stored in the Database Manager. When photos are captured through the UI Controller, they are uploaded to Cloudinary for storage and processing. When new issue reports are created, the system retrieves association contact information from the database and sends email notifications via SendGrid.

The Database Manager serves as the central data persistence layer, implemented using PostgreSQL for persistent storage and Redis for caching. It receives data from the Session Manager, Geo/Map Controller, and Resident Profile components. All user data, building records, reported issues, and associated information are stored in the database to maintain system state and enable retrieval of historical data.

For detailed dependency information, see Dependency_Description.md.

3.3 Interface Description

3.3.1 UI Controller Component

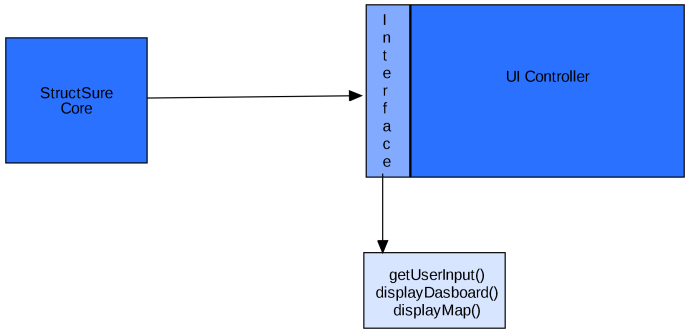

Figure 4: UI Controller Interface

Figure 4 above shows the UI Controller interface. The UI Controller communicates with the StructSure Core via HTTP/HTTPS requests through the Python API wrapper, routing user actions, photo data, location information, and report details.

3.3.2 Session Manager and User Authentication

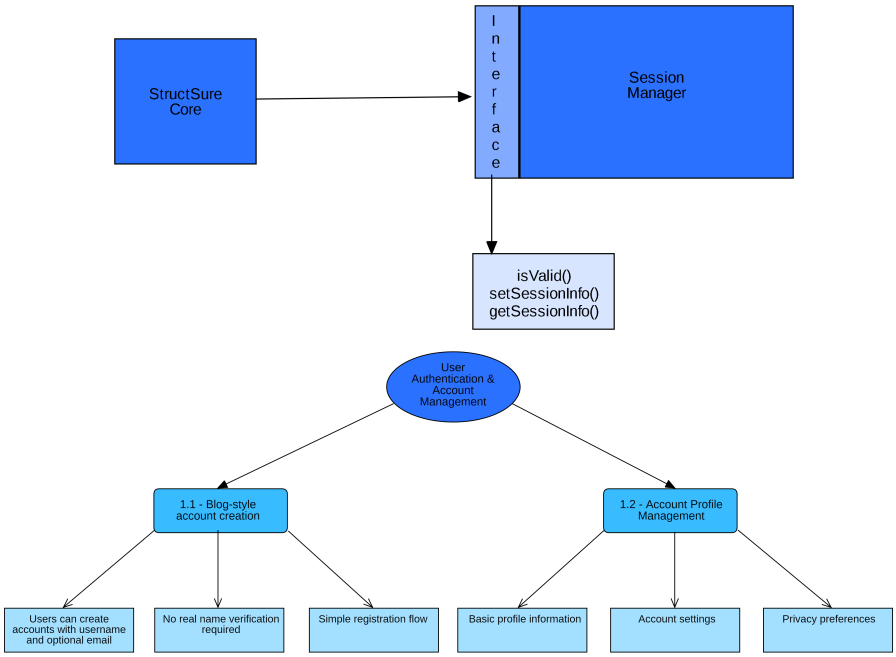

Figure 5: Session Manager and User Authentication Interfaces

Figure 5 above shows the Session Manager authentication interface. The StructSure Core requests authentication status from the Session Manager before allowing system access. Unauthenticated users are redirected to the login screen, and the Session Manager validates credentials throughout the session.

3.3.3 Photo Capture Authentication

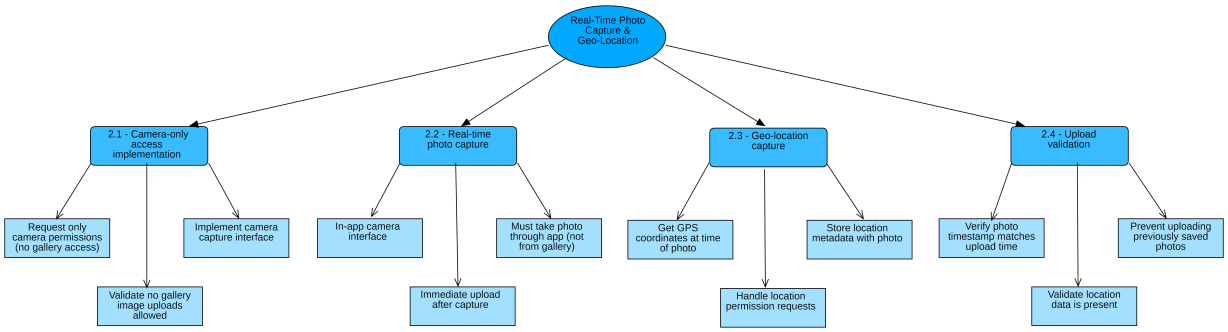

Figure 6: Photo Capture Authentication Interface

Figure 6 above shows the Photo Capture Authentication interface. This interface handles authentication requirements specific to photo capture functionality, ensuring users are properly

authenticated before capturing and uploading photos of infrastructure issues.

3.3.4 Building Identification and Tagging

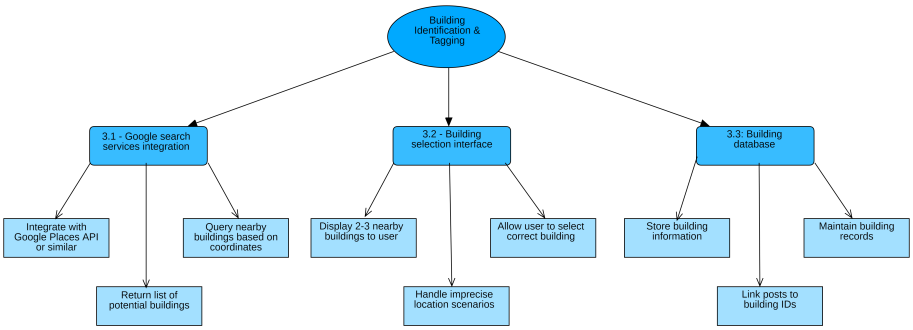

Figure 7: Building Identification Interface

Figure 7 above shows the Building Identification interface. When photos include location data, the system queries nearby buildings and presents 2-3 options for user selection. Selected buildings are stored and associated with reported issues.

3.3.5 Issue Management

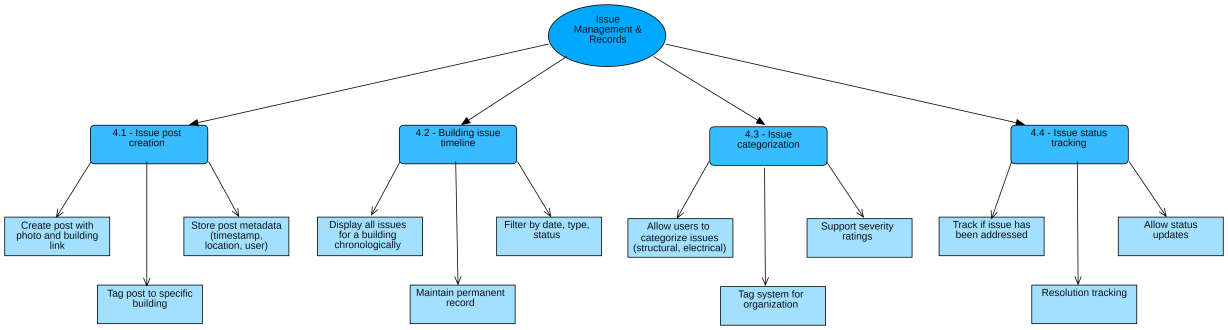

Figure 8: Issue Management Interface

Figure 8 above shows the Issue Management interface. Reports include photo URLs, building associations, GPS coordinates, timestamps, and user information. Reports can be categorized, assigned severity ratings, and tracked for status updates.

3.3.6 Resident Profile

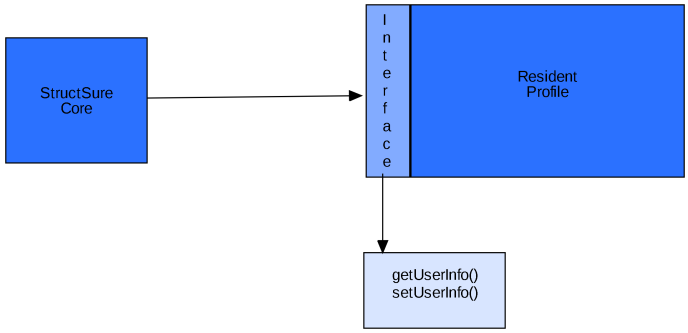

Figure 9: Resident Profile Interface

Figure 9 above shows the Resident Profile interface for managing resident identity, address, association information, and notification preferences. Profiles are linked to reported issues.

3.3.7 Degradation Analyzer

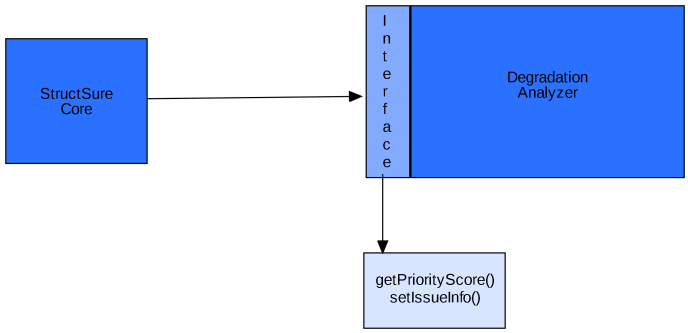

Figure 10: Degradation Analyzer Interface

Figure 10 above shows the Degradation Analyzer interface for assessing issue severity, detecting duplicates, and prioritizing cases. The analyzer interacts with the Resident Profile component to update relevant data.

3.3.8 Association Management

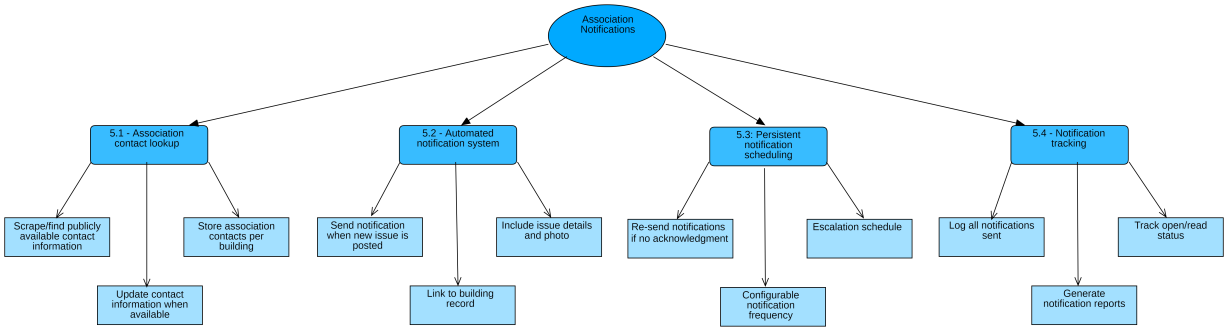

Figure 11: Association Management Interface

Figure 11 above shows the Association Management interface. The current implementation sends email notifications via SendGrid when new issues are reported. Advanced features such as persistent scheduling, escalation workflows, and comprehensive dashboards are outside the scope of this project, as mentioned in Section 2.

3.3.9 Public Visibility

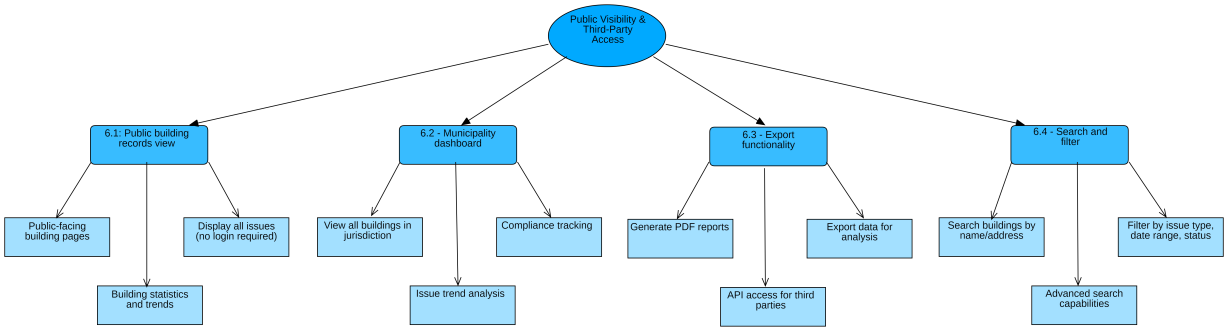

Figure 12: Public Visibility Interface

Figure 12 above shows the Public Visibility interface. Features such as public building records view, municipality dashboards, export functionality, and enhanced search capabilities are outside the scope of the current work but we will aim to add them in this iteration.

3.4 Module Interfaces

Module interfaces define the specific function calls and data structures used for communication between components. Detailed interface specifications will be defined during implementation based on the component relationships described above.

3.5 User Interfaces (GUI)

3.5.1 Start Screen

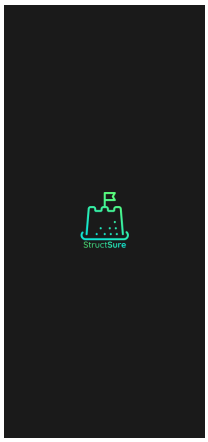

Figure 13: Start/Landing Screen

Figure 13 above shows the start/landing screen of the mobile application. First time and returning users are presented with this screen, which serves as the entry point for authentication and system access.

3.5.2 Homepage

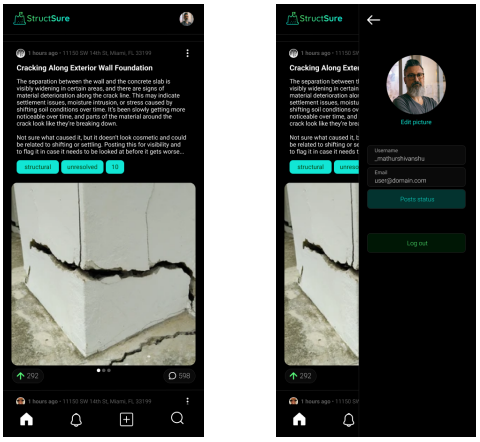

Figure 14: Main Homepage and Homepage with Profile View

Figure 14 above shows the main homepage and homepage with profile view. The main homepage provides navigation to the camera interface, user profile, notifications, and search functionality. The UI Controller presents this dashboard after successful authentication through the Session Manager. The homepage with profile view integrates the Resident Profile component, showing user identity and notification preferences alongside the main navigation.

3.5.3 Photo Capture and Issue Creation

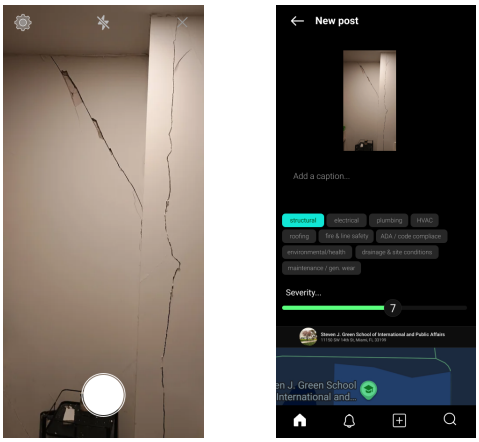

Figure 15: Camera Capture and Issue/Post Creation Screens

Figure 15 above shows the camera capture interface and issue/post creation screen. Users can capture photos of infrastructure issues through the camera screen. Once a photo is captured, the system automatically captures GPS coordinates through the Geo/Map Controller. The photo capture is handled by the Expo SDK camera API as described in Section 2. After capturing a photo with location data, users can categorize the issue (e.g., structural, electrical, plumbing), provide an optional description, and select a severity rating on the post creation screen. This interface interacts with the Reporting API through the StructSure Core to create issue posts associated with buildings.

3.5.4 User Profile Screen

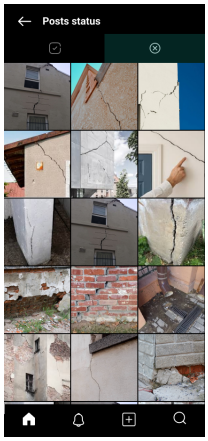

Figure 16: User Profile Screen

Figure 16 above shows the user profile screen. This interface displays the Resident Profile information including identity, address, association information, and notification preferences. Profile data is managed through the StructSure Core and stored in the Database Manager.

3.5.5 Notifications Screen

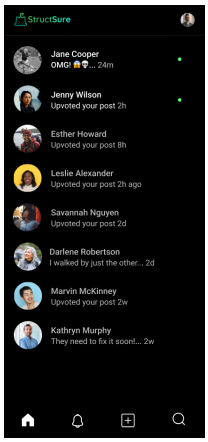

Figure 17: Notifications Screen

Figure 17 above shows the notifications screen. This interface displays notifications related to reported issues, association responses, and system updates. The UI Controller retrieves notification data through the StructSure Core, which manages communication with the notification system.

3.5.6 Search and Building History

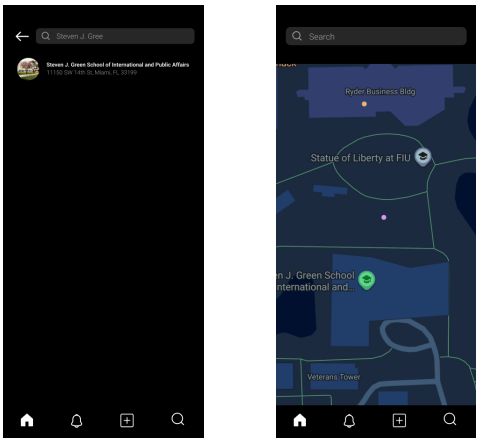

Figure 18: Search by Building and Search by Map Interfaces

Figure 18 above shows the search interfaces. Users can search for buildings by name or address to view associated infrastructure issues, utilizing the Geo/Map Controller and Database Manager to retrieve building information. Users can also explore infrastructure issues by location on a map view, which integrates with the Google Places API through the Geo/Map Controller to display buildings and their associated issues geographically.

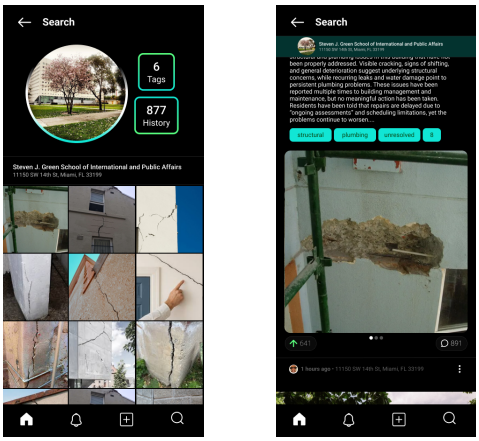

Figure 19: Building History Search and Review Interfaces

Figure 19 above shows the building history search and review interfaces. The search results screen displays reported issues associated with specific buildings, with data provided by the Reporting API through the StructSure Core from the Database Manager. The review interface displays detailed building issue timelines, showing all reported issues for a selected building in chronological order. Each issue displays the photo, timestamp, category, and current status. Users can filter issues by date, category, or status. This functionality is implemented through the Reporting API and retrieves data from the Database Manager.

4 Detailed Design

Sequence Diagrams

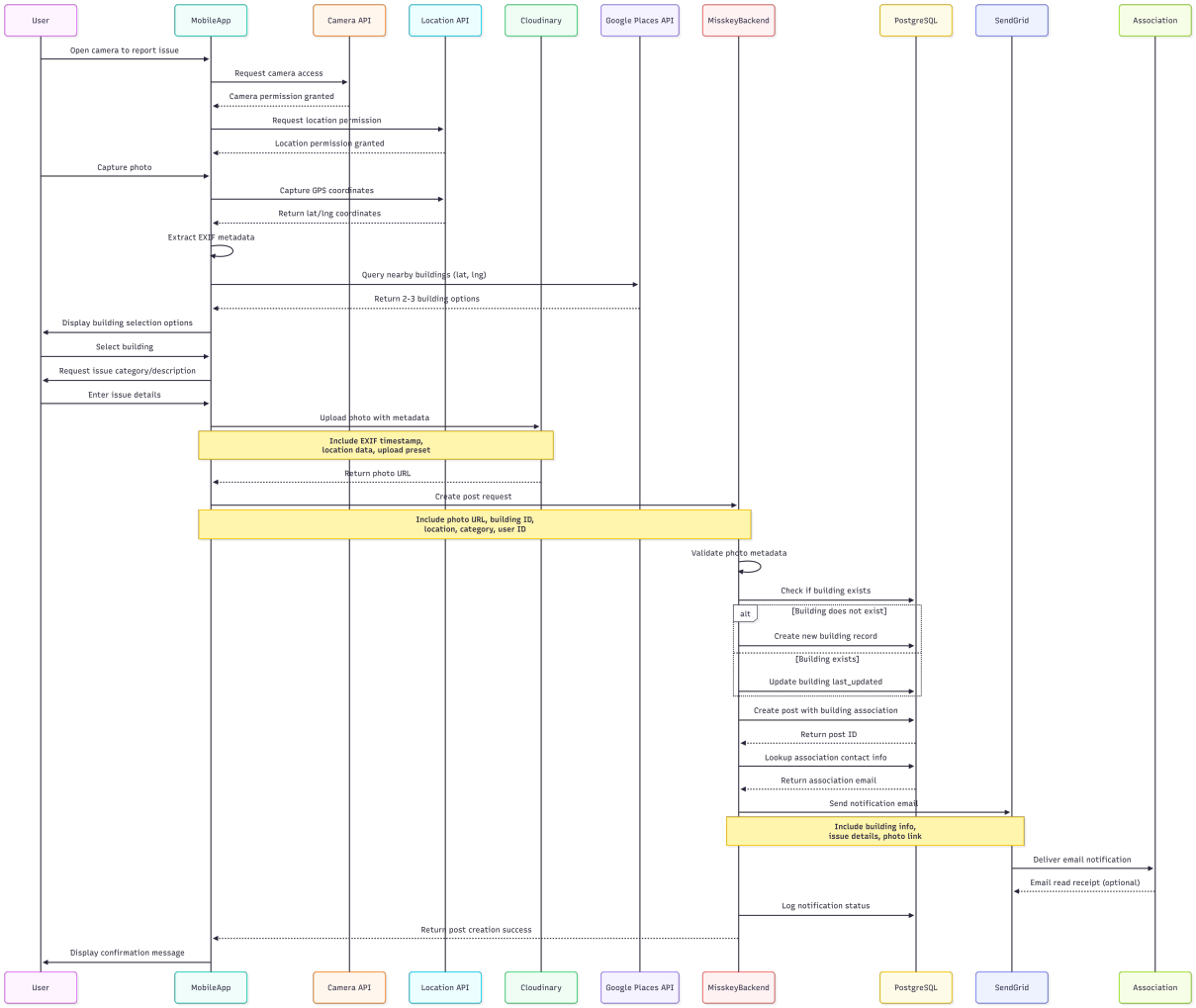

Figure 20: Sequence diagram showing the photo upload and building association workflow

Pseudocode

```
User opens camera interface
Request camera permission from device
If permission granted:
    Display camera preview
    Wait for user to tap capture button
    Capture photo from camera
    Request location permission from device
    If location permission granted:
        Get current GPS coordinates
```

```
        Extract EXIF metadata from photo
        Upload photo to Cloudinary with metadata
        Query Google Places API with GPS coordinates
        Display nearby building options to user
        Wait for user to select building
        Create issue post with photo URL, building ID, and metadata
        Store post in database
        Trigger notification to association
    Else:
        Display error: Location permission required
Else:
    Display error: Camera permission required
```

4.1 Data Detailed Design

The system database extends Misskey's existing schema with the following main tables:

- **Buildings Table**: Stores building information including name, address, GPS coordinates, Google Place ID, and association contact information.
- **Posts Table (Extended)**: Misskey's posts table extended with building association, issue category, severity rating, status, and location data.
- **Notifications Table**: Tracks email notifications sent to associations including recipient, timestamp, delivery status, and retry information.

4.2 RTM

Requirement-ID	Requirement Description	Design Component	Test-Case #
FR-1	User account creation with blog-style registration	Session Manager, User Profile	TBD
FR-2	Camera-only photo capture (no gallery access)	Photo Capture Component	TBD
FR-3	Real-time geo-location capture and tagging	Photo Capture Component, Building Management	TBD
FR-4	Building identification via Google Places API	Building Management Component	TBD
FR-5	Issue post creation linked to buildings	Photo Capture Component, Database Manager	TBD
FR-6	Automated association notifications	Notification System	TBD
FR-7	Public building records view	Public Access Component	TBD
FR-8	Municipality dashboard	Public Access Component	TBD
FR-9	Photo validation and abuse prevention	Photo Capture Component, Validation Module	TBD